Análise de Desempenho: Processamento Concorrente de IDs utilizando POSIX e Win32 Threads

Vitor Hugo Altmann
Ciência da Computação
Universidade Comunitária da Região de Chapecó
Chapecó, SC, Brasil
contato@vitoraltmann.dev

***Resumo*—Este relatório descreve o desenvolvimento e a análise de desempenho de um sistema orquestrador multiprocesso e multithread, construído para consumir IDs de um arquivo, simular o enriquecimento de dados e gravar os resultados em disco de forma segura. Foram avaliadas abordagens sequenciais e concorrentes utilizando mecanismos nativos do sistema operacional (Mutexes e Threads), demonstrando os impactos do paralelismo no tempo total de processamento em diferentes escalas de carga.**

***Index Terms*—Programação Concorrente, POSIX Threads, Win32, Mutex, Desempenho.**

I. Introdução

O processamento de grandes volumes de dados de forma sequencial pode resultar em gargalos significativos de desempenho, especialmente quando as tarefas envolvem operações de entrada e saída (I/O) intensivas, como a consulta a recursos externos e a gravação em disco.

O trabalho propõe a implementação de dois processos: o processo pai (P0), responsável por orquestrar e cronometrar a execução, e o processo filho (P1), que instancia múltiplas threads de trabalho. O objetivo principal é observar o comportamento temporal do programa e identificar os ganhos de desempenho ou possíveis gargalos provocados pela concorrência estruturada.

II. Metodologia

A solução foi desenvolvida na linguagem C, projetada para ser multiplataforma, abstraindo os controladores de *threads* nativos de sistemas baseados em UNIX e Windows.

Para garantir a ausência de condições de corrida, foram implementados dois semáforos binários (Mutexes):

- **Mutex de Leitura:** Garante que duas *threads* não retirem o mesmo identificador do índice da matriz simultaneamente, prevenindo processamento duplicado.
- **Mutex de Escrita:** Ordena a gravação no arquivo de log final para impedir que as saídas em texto sejam mescladas, sobrepostas ou corrompidas no arquivo de destino.

O processamento computacional intensivo foi mantido fora das seções críticas para maximizar o uso do paralelismo da CPU. Foram executados três testes com listas de tamanhos distintos (Pequena: 100 itens; Média: 1.000 itens; Grande: aproximadamente 6.000 itens), comparando a execução isolada com a carga distribuída entre 16 trabalhadores.

III. Resultados e Análise de Desempenho

Os dados de tempo total extraídos pelo orquestrador durante os testes estão dispostos na Tabela I.

Tabela I
Comparativo de Desempenho (Tempo Real)

Carga (Tamanho)	1 Thread (s)	16 Threads (s)	Status
Pequena (100)	0.105	0.416	Concluído
Média (1.000)	0.158	0.383	Concluído
Grande (>6.000)	0.361	0.806	Concluído

A. Discussão

A análise dos tempos revela que a execução sequencial (1 *thread*) obteve desempenho superior em todos os cenários testados, finalizando a carga grande em apenas 0.361s contra 0.806s da versão paralela. O uso de paralelismo não ofereceu ganhos devido à extrema contenção gerada pelas seções críticas. Como a gravação em log e a leitura do índice exigem exclusão mútua, as 16 *threads* passaram a maior parte do ciclo de vida em estado de espera (bloqueadas pelos *mutexes*), gerando um alto custo de chaveamento de contexto (*overhead*). O experimento demonstra que introduzir concorrência em rotinas que dependem massivamente de I/O serializado pode gerar degradação de desempenho.

IV. Conclusão

A abstração de *threads* e exclusão mútua provou ser eficaz para garantir a segurança dos dados. Contudo, o processamento concorrente confirmou que

a contenção por recursos compartilhados (arquivos) anula os ganhos da paralelização. A implementação justifica-se em cenários de processamento assíncrono de CPU intensivo, mas exige cautela quando o gargalo principal é o acesso sincronizado a disco.